Nexus Dev ABP 3

Validação do Fluxo Lead → Negociação → Conversão

Durante o desenvolvimento do projeto ABP, foi realizada uma Task referente a uma análise com o objetivo de validar o funcionamento do fluxo principal: Lead → Negociação → Conversão. Esta análise considerou tanto a estrutura do banco de dados quanto a implementação das camadas de backend (services, repositories, controllers e rotas).

01- Análise da Modelagem de Dados

Inicialmente foi analisado o arquivo ***schema.prisma***, responsável pela definição das entidades do sistema.

A entidade Lead está definida da seguinte forma:

```
model Lead {
  id          String     @id @default(uuid())
  origin      LeadOrigin
  clientId    String     @map("client_id")
  userId      String     @map("user_id")
  teamId      String     @map("team_id")
  storeId     String     @map("store_id")
  createdAt   DateTime   @default(now()) @map("created_at")
  updatedAt   DateTime   @updatedAt @map("updated_at")

  // Relações
  client      Client       @relation(fields: [clientId], references: [id])
  user        User         @relation(fields: [userId], references: [id])
  team        Team         @relation(fields: [teamId], references: [id])
  store       Store        @relation(fields: [storeId], references: [id])
  negotiation Negotiation?

  @@map("leads")
}
```

Observa-se que o Lead possui relacionamento opcional com Negotiation, indicando que um lead pode ou não possuir uma negociação associada.

```
model Negotiation {
  id         String                 @id @default(uuid())
  leadId     String                 @unique @map("lead_id")
  importance NegotiationImportance  @default(FRIO)
  status     NegotiationStatus      @default(ABERTA)
  reason     String?
  createdAt  DateTime               @default(now()) @map("created_at")
  updatedAt  DateTime               @updatedAt @map("updated_at")

  lead    Lead                  @relation(fields: [leadId], references: [id])
  history NegotiationHistory[]

  @@map("negotiations")
}
```

O campo "leadId" é único, o que garante uma relação de 1:1 entre Lead e Negotiation. Isso indica que cada lead pode possuir no máximo uma negociação, o que é coerente com o fluxo proposto.

Conclusão da modelagem:
A estrutura de banco de dados está adequada e contempla o fluxo Lead → Negociação. No entanto, não há nenhum campo que represente diretamente o status do Lead (ex: novo, em negociação, convertido).

02- Análise da Implementação de Leads

A camada de serviço responsável pelos leads foi analisada:

```
async create(data: {
  origin: string;
  clientId: string;
  userId: string;
  teamId: string;
  storeId: string;
}): Promise<Lead> {
  return leadRepository.create({
    origin: data.origin as any,
    client: { connect: { id: data.clientId } },
    user: { connect: { id: data.userId } },
    team: { connect: { id: data.teamId } },
    store: { connect: { id: data.storeId } },
  });
}
```

Observa-se que a criação do lead está corretamente implementada, conectando todas as entidades necessárias.

Além disso, o controller expõe os endpoints:

```
TS leadRoutes.ts ×
1   import { Router } from 'express';
2   import leadController from '../controllers/LeadController';
3
4   const router = Router();
5
6   router.get('/', leadController.index);
7   router.get('/:id', leadController.show);
8   router.post('/', leadController.store);
9   router.put('/:id', leadController.update);
10  router.delete('/:id', leadController.destroy);
11
12  export default router;
```

Conclusão:
A funcionalidade de Lead está completa, permitindo todas as operações básicas (CRUD).

03- Análise da Negociação

Apesar de existir no banco de dados, ainda não foram criados os trechos de código relacionados à negociação nas camadas de aplicação em breve vamos construir.

- NegotiationService
- NegotiationController
- Rotas para negociação

Isso significa que, embora a estrutura exista, não há forma de manipular negociações via API.

Impacto no sistema:
O fluxo é interrompido após a criação do Lead, pois não é possível iniciar ou acompanhar negociações.

04- Conversão de Lead

As próximas etapas do projeto envolvem a criação de endpoint ou lógica que realize a conversão de um lead em cliente.

Exemplo do que NÃO existe no sistema:

POST /leads/:id/convert

E construir endpoint que:

- Atualize o status do lead
- Finalize a negociação automaticamente
- Relacione a conversão com a entidade Client

Conclusão:
A etapa final do fluxo (conversão) não está implementada.

05- Desenvolver o Controle de Estado

Outro ponto importante identificado é a ausência de controle de status do lead.

Atualmente, o modelo Lead não possui nenhum campo como:

status: "NOVO" | "EM_NEGOCIACAO" | "CONVERTIDO"

Impacto:
Não é possível rastrear em qual etapa do fluxo o lead se encontra.

06- Conclusão Geral

A análise demonstra que o sistema possui uma base sólida para gerenciamento de leads, com modelagem bem estruturada e implementação funcional da camada de Lead.

No entanto, o fluxo principal do sistema encontra-se desenvolvimento, pois:

- A negociação não está implementada na camada de aplicação
- Não existe funcionalidade de conversão
- Não há controle de status do lead

Fluxo atual identificado:

Lead → (fluxo interrompido)

Fluxo esperado:

Lead → Negociação → Conversão

07- Próximos passos de desenvolvimento da ABP

Para completar o fluxo vamos:

1. Implementar NegotiationService, Controller e Routes
2. Criar endpoint para conversão de leads
3. Adicionar campo de status no modelo Lead
4. Definir regras de negócio para:
    - criação de negociação
    - encerramento
    - conversão